Bachelor Project

3D Accelerometer Evaluation and ST Core Power Management

Waleed Rajab

October 09, 2025

Submitted to the University of Freiburg
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

universität freiburg

University of Freiburg
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

Author Waleed Rajab,
Matriculation Number: 4772281

Editing Time May 25, 2025 - October 09, 2025

Examiners Prof. Dr. Thomas Stieglitz,
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

Supervior Prof. Dr. Thomas Stieglitz,
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

Declaration I hereby declare, that I am the sole author and composer of this Thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I hereby also declare, that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Place, Date

Signature

Abstract

This study evaluates three accelerometer modules Bosch BMI270, LSM6DSOX, and MIS2DH integrated with an STM32 microcontroller to identify a low power, 1 kHz motion sensing baseline for arm movement triggering. Each sensor was configured near 1 kHz with FIFO enabled and communicated over I^2C in different modes. Methods included GPIO-toggled timing verified on a PicoScope, current measurement at the sensor V_{DD}, and angle estimation via a complementary filter that blends accelerometer and gyroscope data. Results show that increasing bus speed generally raises effective sampling rate while reducing or stabilizing current draw, BMI270 current decreased from 2.22 mA to 1.41 mA with measured rates rising into the 1.7 kHz range, LSM6DSOX 0.60–0.63 mA with ∼1.7 kHz effective rates, MIS2DH drew 1.15–1.25 mA and reached up to 1.6 kHz at 1 MHz. All three sensors can meet 1 kHz under faster I^2C modes, with LSM6DSOX offering the best current for rate trade off for real-time, low-power applications. The work provides practical setup guidancebus mode, FIFO use, and fusion choices for embedded inertial sensing on STM32.

Contents

1 Introduction **1**

2 Theoretical Background **3**

2.1 Sampling latency and buffering 3

2.2 I^2C bus modes 3

2.3 Complementary filter 4

2.4 Power management on STM32 4

3 State of the Art **5**

4 Implementation **7**

4.1 Overall Architecture 7

4.2 Hardware Setup 8

4.3 Sensor Configuration 9

4.4 Firmware and Software Design 10

5 Experimental Setup **11**

5.1 Energy Consumption 11

5.2 Sampling 12

6 Results and Discussion **15**

6.1 Energy consumption and samling rate 15

6.1.1 BMI270 15

6.1.2 LSM6DSOX 16

6.1.3 MIS2DH 16

6.1.4 Difference between measured and configured sampling rate 18

6.2 Discussion 18

6.2.1 Current and sampling rate with different sensors 18

7 Summary **21**

8 Acknowledgments **23**

Bibliography **25**

. **28**

A.0.1 The external device with the headpiece. 28

A.0.2 Live data for emulating arm movement 29

Symbolslist

N_{FIFO} Samples waiting in the FIFO.
P_{total} Total power reported in results tables.
T_{delay} End to end delay.
T_{proc} Processing time on the host MCU.
T_{sensor} Internal sensor conversion time.
T_{transfer} Bus communication/transfer time.
Δt Sampling interval.
α Complementary filter blend factor.
$\hat{\theta}_k$ Estimated angle at sample k.
ω_k Gyroscope angular rate at sample k.
τ Complementary filter time constant.
$\theta_{\text{acc},k}$ Accelerometer derived angle at sample k.
f_s Sampling frequency.
f_{sample} True sampling frequency.
f_{toggle} Measured GPIO toggle frequency.
P Energy consumption.

Figure Index

4.1 Illustration of different stages the motion data traverses 7
4.2 Hardware setup with accelerometer sensor connected to the STM32 microcontroller. 8
4.3 Schematic of the used pull-up resisors with LSM6DSOX. 9

5.1 setup for current consumption. 12
5.2 setup for sampling measurement. 13

6.1 frquency result vs sensor set 17
6.2 Measured frequency vs. I^2C mode 17
6.3 Configured set vs. measured sampling frequency. 18

A.1 external device 28
A.2 Live stream with complementary filter. 29
A.3 Live stream without complementary filter. 30

Chapter 1

Introduction

Embedded systems are getting smaller, smarter, and more energy aware. This makes it possible to build devices that sit close to the human body and react in real time. Motion sensing is central to this idea because it turns movement into data that a microcontroller can process. Consequently, three-dimensional orientation dynamics are of particular relevance in biomedical and assistive applications, where high-accuracy sensing must be achieved under stringent energy constraints. In this work a motion sensing units shall be used to help a patient move an arm that cannot be moved voluntarily. The study uses an external device worn on the biceps. This setup stands in for a future external device in which the sensor would be integrated, detect intended motion, and trigger electrical stimulation so the arm moves A.1.

Three different sensors were evaluated to learn which one offers the best mix of energy use, size, and functional behavior during arm movement emulation. Each sensor was connected to an STM32 microcontroller over I^2C and exercised in different speed modes. On sensor FIFO buffering was enabled to collect samples in bursts and to reduce bus traffic and timing variation. In a second step the STM32 microcontroller filters the angle estimation. Using a complementary filter that blends accelerometer and gyroscope data 2.3. This provided a smooth and timely angle signal that is stable and quick enough to be used for a closed-loop controller that can produce natural looking motions using neuro stimulation.

On the microcontroller side, the focus was power management on a STM32. Clock settings were adjusted and their effect on current consumption and latency was measured. The aim is to meet the timing needs of angle tracking and stimulation while staying within the energy budget expected for a device that could later be implanted.

The goal of this work is to build a practical sensing and processing baseline for

an implant based stimulator that trigger arm movement for users with limited mobility.

Chapter 2

Theoretical Background

2.1 Sampling latency and buffering

Accurate data flow of sensor depends on consistent timing. The sampling frequency f_s must be high enough to capture natural arm motion without aliasing, while the sample interval $\Delta t = 1/f_s$ should remain as constant as possible so that filters and attached controllers behave predictably.
Many inertial sensors include a FIFO buffer. Instead of sending or providing every sample separately, the sensor stores several samples and transfers or provide them in a block. This reduces the constant communication load on the microcontroller into longer but less frequent bursts. Longer gaps in between the data reception and processing allows the controller to sleep for longer periods which therefore reduces the amount of energy consumed.
The total end to end delay from motion to processed data can be approximated as

$$T_{\text{delay}} \approx T_{\text{sensor}} + \frac{N_{\text{FIFO}}}{f_s} + T_{\text{transfer}} + T_{\text{proc}},$$

where T_{sensor} is the internal sensor conversion time, $\frac{N_{\text{FIFO}}}{f_s}$ is the waiting time in the FIFO, T_{transfer} is the bus communication time, and T_{proc} is the processing time on the host.

2.2 I^{2}C bus modes

Data is transfered from the sensor to the microcontroller over I^{2}C. Three standard modes are relevant: Standard mode at 100 kHz, Fast mode at 400 kHz, and Fast mode Plus at 1 MHz. Higher data rates shorten transfer time and can lower

energy per sample when paired with FIFO reads, but they demand better signal integrity and careful timing. A similar Implementation in all three modes allows a fair comparison of throughput, delay, and energy for the three sensors.

2.3 Complementary filter

A complementary filter combines two imperfect and noisy signals so their strengths cover each other's weaknesses [1]. For orientation, the gyroscope reacts quickly but drifts over time, while the accelerometer is stable in the long run but noisy during fast motion. The filter blends them so high-frequency changes come mostly from the gyroscopic sensor and low-frequency corrections come from the accelerometer. With sampling time Δt, gyro rate ω_k, and accelerometer angle $\theta_{\mathrm{acc},k}$,

$$\hat{\theta}_k = \alpha\big(\hat{\theta}_{k-1} + \omega_k\,\Delta t\big) + (1-\alpha)\,\theta_{\mathrm{acc},k}, \qquad \alpha = \frac{\tau}{\tau + \Delta t}.$$

Here $\hat{\theta}_k$ is the fused angle, $\alpha \in (0,1)$ is the blend, and τ is a time constant that sets the split between "fast" and "slow"(see comparison in figure A.0.2).

2.4 Power management on STM32

Energy use comes from both the sensor and the microcontroller. On a STM32, the key factors are the system clock, peripheral clocks, and sleep modes. Running the core slower cuts power, and with DMA and FIFO the processor can sleep between data bursts. The aim is to keep sampling and data flow consistent while keeping power consumption low.

Chapter 3

State of the Art

Recent work shows that stable sampling and efficient data handling are important for motion sensing and similar applications [2]. A common theme is the balance between saving energy and keeping sample delays low and consistent. Many studies report that buffering, where sensor data is collected before it is sent, can reduce communication and processor activity. This lowers power use and makes sampling more regular [3].

However, buffering also introduces delay, because the data has to wait before it is transferred and processed. Several authors point out that this trade off must be considered carefully [4]. While buffering helps save energy, it can also increase the end to end latency, which in real time systems may reduce responsiveness and affect the user experience.

To deal with this, other research has looked at ways to keep the timing correct even when buffering is used [5] [6]. Examples include calculating the actual sampling interval afterwards or adding timestamps to each measurement. These methods make it possible to keep the data flow consistent, while still enjoying the benefits of lower communication load and reduced power consumption use.

Chapter 4

Implementation

This chapter describes the practical implementation of the proposed system. It explains the hardware setup, software design, and data flow. The goal is to show how the concepts from the theoretical background and state of the art were applied in practice. This section introduces the overall architecture of the implemented system. The design is divided into five main components: the sensor, the STM32 microcontroller, the I^2C bus, the processing layer, and the output application. The sensor captures motion data and delivers it to the microcontroller. The STM32 manages sampling, communication, and data transfer to the processing stage. The processing layer formats and interprets the data, and the final results are then forwarded to the output application for storage, visualization, or further use. (Figure 4.1 illustrates this flow.)

4.1 Overall Architecture

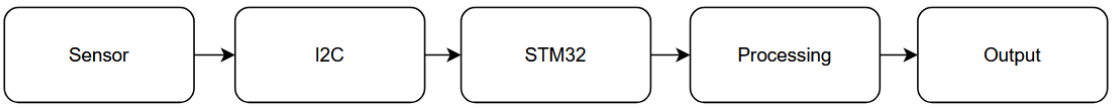


Figure 4.1: Illustration of different stages the motion data traverses

4.2 Hardware Setup

The hardware setup forms the foundation of the project. It consists of an accelerometer sensor with a gyroscope, a STM32 microcontroller board, and a host computer. The sensor measures both acceleration and rotation, transmitting this information to the microcontroller through a communication interface.
The STM32 takes care of talking to the sensor, collecting the measurements, and passing them on. It is powered through USB, which also provides a direct serial link to the computer. The computer is used to log the data and display it during testing.
A illustration of the full setup is shown in Figure 4.2. The sensor module is connected to the STM32 board with jumper wires, and the board itself is connected to the computer with a USB cable. This makes the system compact, easy to set up, and flexible for experiments.

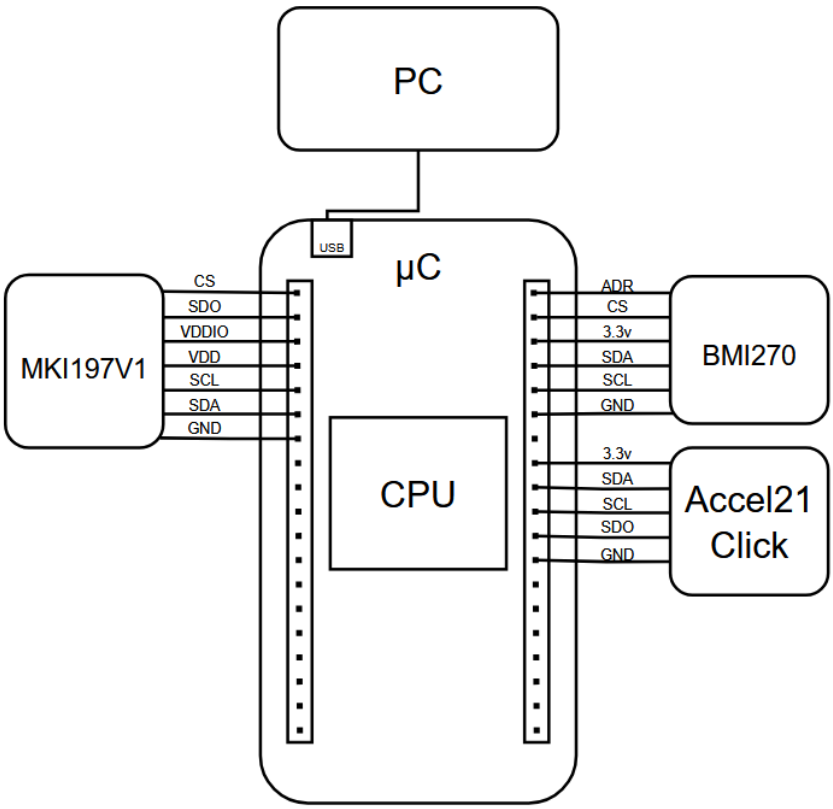


Figure 4.2: Hardware setup with accelerometer sensor connected to the STM32 microcontroller.

In this project, three different accelerometer modules were used. The first was the Bosch BMI270, a low power 6-axis [7]. The breakout board already included pull-up resistors for the I^2C lines, which made it possible to connect the sensor directly to the STM32 without additional components.
The second device was the LIS2DH, a 3-axis accelerometer [8]. This module also provided onboard pull-up resistors on the I^2C bus, so it could be interfaced with the microcontroller in the same way.
The third module was the LSM6DSOX, a 6-axis with extended functionality[9]. Unlike the other two boards, it did not include pull-ups on the I^2C lines. To ensure reliable communication, external resistors of 4.7 kΩ were therefore added between SDA and 3.3 V, and between SCL and 3.3 V as shown in figure 4.3.
All accelerometer modules were supplied with 3.3 V and connected to the STM32 using short jumper wires. For this experimental setup, a dedicated PCB was not required, since the short connections ensured reliable I^2C communication even at higher bus speeds. Each module was also equipped with local decoupling capacitors to stabilize the supply voltage and reduce noise.

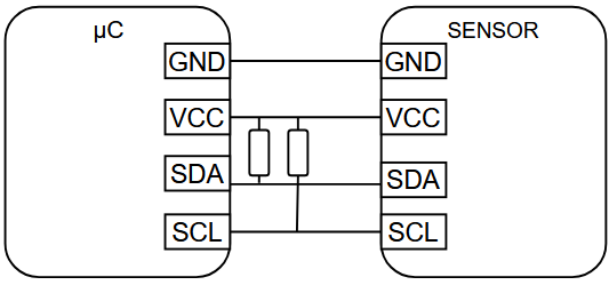


Figure 4.3: Schematic of the used pull-up resisors with LSM6DSOX.

4.3 Sensor Configuration

To keep the comparison fair and the code simple, each sensor was initialized with a predefined configuration before any measurements. The output data rate could not be set to exactly 1 kHz on all devices, so the closest available rate was selected for each sensor.

LSM6DSOX Both accelerometer and gyroscope were set to an output data rate of $\sim 1.66\,\text{kHz}$. The accelerometer range was configured to $\pm 2\,g$ and the gyroscope to $\pm 2000\,\text{dps}$. The on-chip FIFO was enabled in *continuous* mode with timestamp batching and a small watermark (8 samples) [9], allowing block-wise reads to reduce I^2C traffic while keeping latency low.

BMI270 Using the Bosch driver API [10], the accelerometer ran at $1600\,\text{Hz}$ with $\pm 2\,g$ full scale. The gyroscope was configured to $1600\,\text{Hz}$ at $\pm 2000\,\text{dps}$, with performance-optimized filtering and noise settings. In short, the BMI270 was tuned for stable, low-noise data at roughly the same rate as the LSM6DSOX.

MIS2DH The device was seted in high-resolution mode with block data update enabled. The full-scale was typically $\pm 2\,g$ with all three axes enabled.

4.4 Firmware and Software Design

The firmware on the STM32 is responsible for setting up the sensor, keeping the sampling stable, and sending the collected data to the computer. When the system starts, the firmware initializes the communication and configures the sensor with the desired measurement ranges and sampling rate. This makes sure the sensor works in a consistent and reliable way.
After that, the microcontroller takes over the measurement process. The sensor produces motion data continuously, and the firmware reads it at regular intervals over the I^2C bus.
Before sending the results to the computer, the firmware does some light processing: it formats the raw values, and prepares the data for transfer to the host computer. The data then goes over the USB serial link to the computer. On the computer, a simple application logs the measurements and provides a live view during testing over UART protocol. This way, the microcontroller only handles the real-time tasks, while the computer handles storage and analysis.
Finally, a complementary filter fuses the high-frequency, gyro-based dynamics with the low-frequency, accelerometer based orientation to deliver a drift resistant, low noise angle estimate in real time.

Chapter 5

Experimental Setup

The experimental part of this works is divided into two main parts. The first part focuses on measuring the energy consumption of the system under different operating conditions. The second part evaluates the system performance during continuous operation with a sampling rate of 1 kHz, while emulating arm motion. Together, these experiments provide insights into the efficiency and the functional behavior of the implemented setup.

5.1 Energy Consumption

One of the main goals of this work was to compare the three accelerometer modules in terms of their energy use. For this reason, the experiments focused on measuring and comparing the power consumption of each module under realistic operating conditions.
A hand multimeter was used for all measurements [11]. To avoid LED loading on the breadboard, the LEDs were removed before taking measurements. The multimeter was connected directly to the VDD line of the sensor modules during operation (see Fig. 5.1). This setup made it possible to observe the current drawn by each sensor in real time under the same supply voltage. The procedure was repeated for all three modules so that the results could be compared fairly and related to the specific design of each sensor.

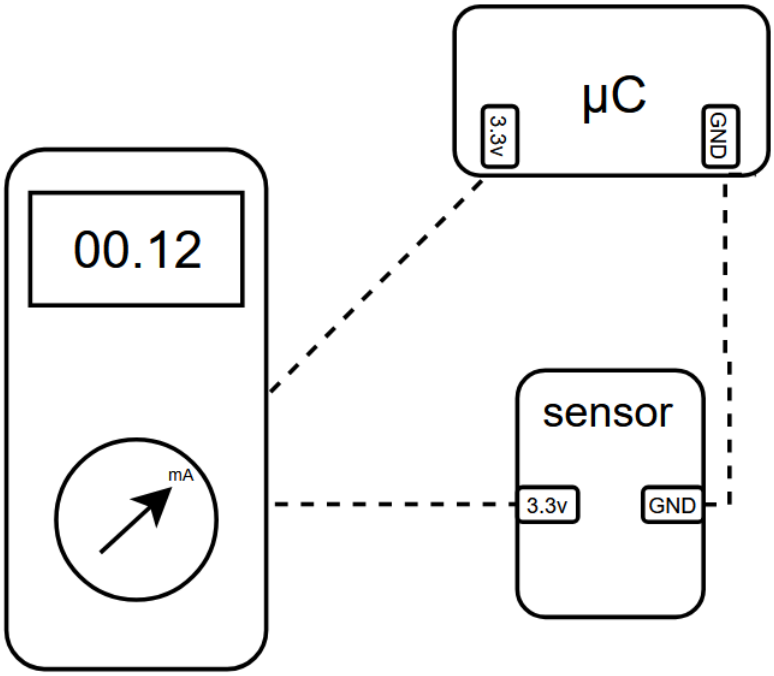


Figure 5.1: setup for current consumption.

5.2 Sampling

The sensors were tested with I^2C in different speed mode to see how bus speed affects data transfer and timing behavior 2.2. To achieve a final sampling rate of 1 kHz, each sensor was configured to achieve $\sim$1 kHz as described in Section 4.3. The effective sampling rate was measured with a PicoScope 2408B [12]. Channel 1 was connected to microcontroller Pin 6 (Figure 5.2). This pin toggled each time new accelerometer data was acquired, allowing the Picoscope 2408B to measure the resulting square-wave frequency [12]. Because a full period requires both a low and a high level, the scope reports half the sampling frequency. In the same motion-processing routine, the raw sensor data are read, the angle is computed, and a complementary filter is applied (Section 2.3).

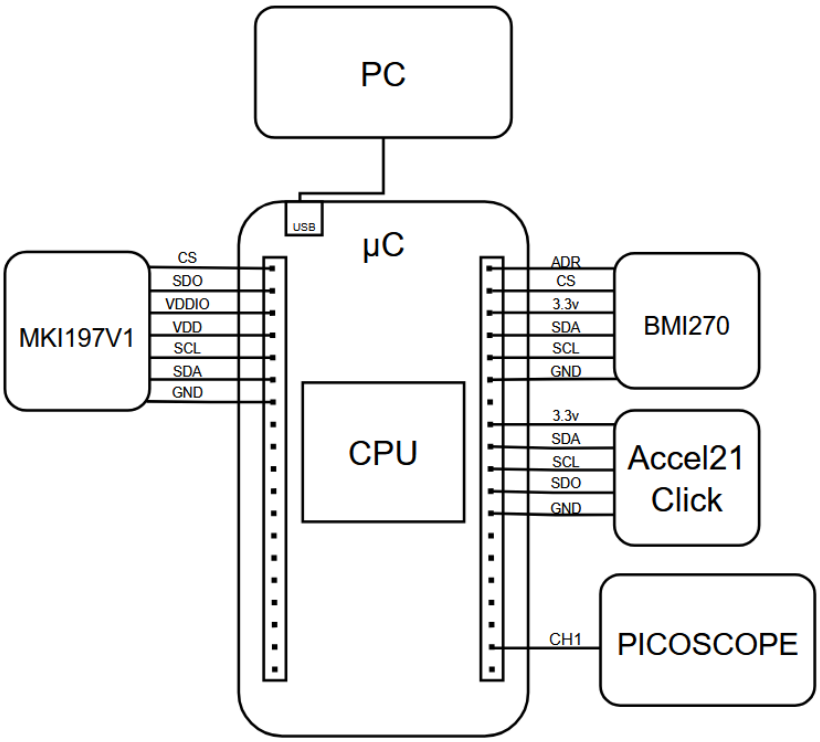


Figure 5.2: setup for sampling measurement.

Chapter 6

Results and Discussion

This chapter shows the measurement result for the three accelerometer modules at different I^2C speeds. For each sensor was recorded the supply current and the effective sampling frequency by the PicoScope. Since the pin toggles once per sample, the measured toggle frequency corresponds to half the sampling rate and must be multiplied by two to obtain the full rate.

6.1 Energy consumption and samling rate

6.1.1 BMI270

Table 6.1: BMI270: current and measured frequency with different I^2C mode

I^2C mode	Bus speed / kHz	Current / mA	measured frequency / Hz	P_{total}/mW
Standard	100	2.22	363	7.326
Fast	400	1.92	744	6.336
Fast Plus	1000	1.41	868	4.653

The BMI270 current drops from 2.22 mA at 100 kHz to 1.41 mA at 1 MHz (Figure 6.1). The measured frequency rises from 363 Hz to 744 Hz when moving to Fast mode, then settles at 868 Hz in Fast Plus (Figure 6.3).

6.1.2 LSM6DSOX

Table 6.2: LSM6DSOX: current and measured frequency with different I^2C mode

I^2C mode	Bus speed / kHz	Current / mA	measured frequency / Hz	P_{total}/mW
Standard	100	0.602	3	1.987
Fast	400	0.611	851	2.016
Fast Plus	1000	0.626	855	2.066

The LSM6DSOX stays in the 0.60–0.62 mA range across modes (Figure 6.1) and reaches the highest measured frequencies at faster bus speeds with 855 Hz (Figure 6.3).

6.1.3 MIS2DH

Table 6.3: MIS2DH: current and measured frequency with different I^2C mode

I^2C mode	Bus speed / kHz	Current / mA	measured frequency / Hz	P_{total}/mW
Standard	100	1.250	500	4.125
Fast	400	1.200	650	3.960
Fast Plus	1000	1.150	800	3.795

MIS2DH consumed about 1.15–1.25 mA and its current decreases slightly as the bus speed increases (Figure 6.1), while the measured frequency still rises with speed with maximum of 800 Hz (Figure 6.2).

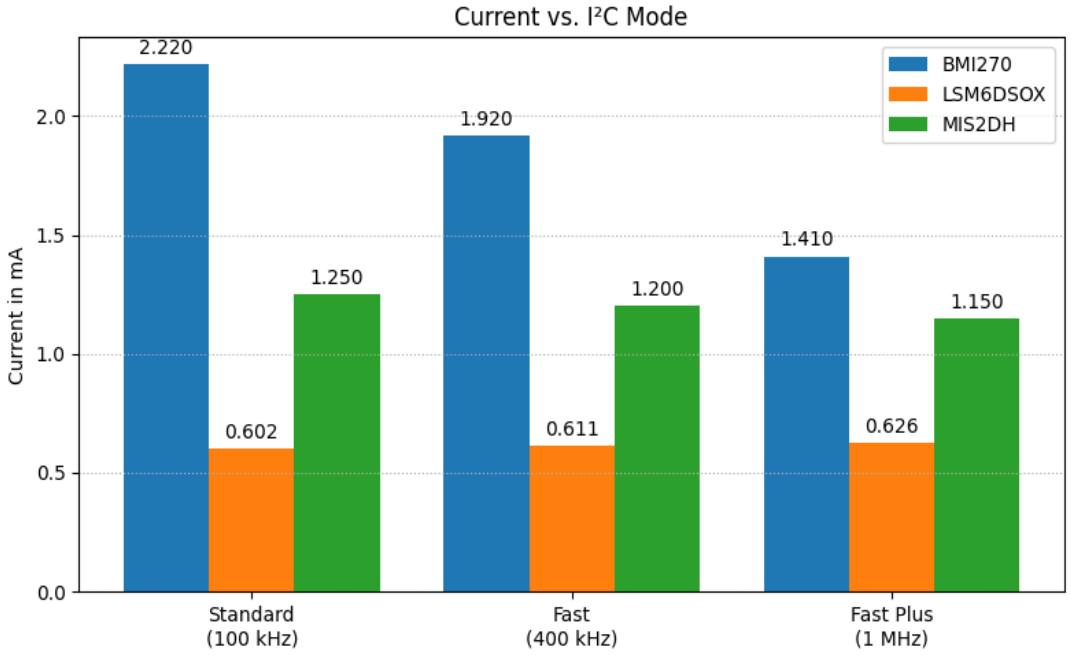


Figure 6.1: frquency result vs sensor set

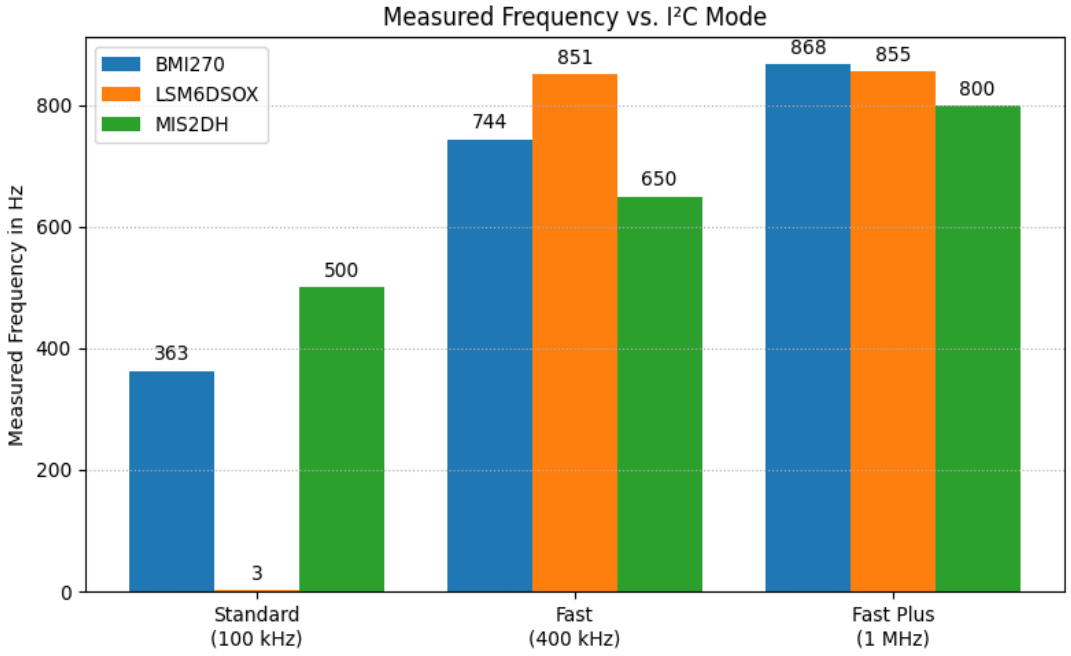


Figure 6.2: Measured frequency vs. I^2C mode

6.1.4 Difference between measured and configured sampling rate

Figure 6.3 compares the configured output-data rates with the measured sampling rates. Each sensor was set to the nearest available to 1 kHz (see Section 4.3). LSM6DSOX: set 1660 Hz, measured 1710 Hz. BMI270: set 1600 Hz, measured 1736 Hz. MIS2DH (high-resolution): measured 1600 Hz. Small deviations are expected due to internal clock tolerances, FIFO/interrupt handling, and the GPIO-toggle measurement method used with the PicoScope.

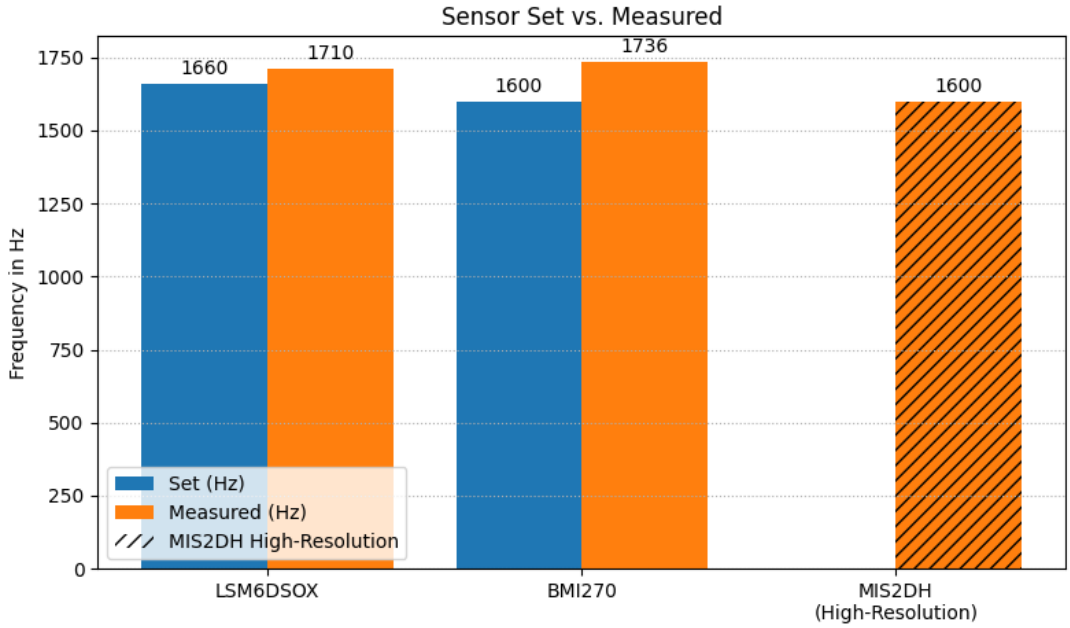


Figure 6.3: Configured set vs. measured sampling frequency.

6.2 Discussion

6.2.1 Current and sampling rate with different sensors

Using the GPIO-toggle method, the PicoScope measures half the sampling rate, so all frequencies are reported as $f_{\text{sample}} = 2\, f_{\text{toggle}}$. With this correction, the BMI270 shows a current drop from 2.22 mA (100 kHz) to 1.41 mA (1 MHz) while the effective rate rises from 726 Hz to 1,736 Hz. The LSM6DSOX holds a nearly constant current around 0.60–0.63 mA and reaches ≈ 1,700 Hz at 400 kHz and 1 MHz. The MIS2DH draws 1.25–1.15 mA across speeds and achieves 1,000, 1,300

and 1,600 Hz in Standard, Fast, and Fast Plus modes, respectively. In summary, after applying the 2× correction, all sensors meet or exceed the $\sim 1\,\text{kHz}$ target in the 400 kHz and 1 MHz modes, with LSM6DSOX offering the lowest current for a given effective rate and BMI270 showing the strongest rate increase as bus speed rises.

Chapter 7

Summary

This work compared three accelerometer modules (BMI270, LSM6DSOX, MIS2DH) for triggering arm movement in an emulated I^2C setup (see Figure A.2). With this correction, BMI270 current dropped from 2.22 mA at 100 kHz to 1.41 mA at 1 MHz, and the effective sampling rate rose from about 726 Hz to 1,736 Hz, meeting the $\sim$1 kHz target in the faster modes. LSM6DSOX kept current low and stable around 0.60–0.63 mA and reached about 1.7 kHz at 400 kHz and 1 MHz. One 100 kHz reading (5.6 Hz) appears to be an outlier. MIS2DH drew 1.15–1.25 mA and achieved 1,000, 1,300, and 1,600 Hz across the three modes. All measurements were taken on breakout boards with jumper wires and on-board pull-ups, so a dedicated PCB with tuned pull-ups and stronger decoupling could shift the exact numbers. Overall, all three sensors can reach $\approx$1 kHz after the 2$\times$ correction, with LSM6DSOX offering the lowest current for a given rate. For fast, reliable motion triggering under tight power limits, LSM6DSOX is the recommended choice. BMI270 is a good alternative if slightly higher current is acceptable, and MIS2DH performs adequately in high-resolution mode with moderate current. Overall, the LSM6DSOX achieved the best results and is recommended.

Chapter 8

Acknowledgments

I would like to than my supervisor Martin März as well as Prof. Dr. Stieglitz for giving me the opportunity to conduct the research required for this work at the University of Freiburg. In addition, I would like to thank Christian Stolle and the people working in the Laboratory in CorTec for helping me with using components for the project.

Bibliography

[1] seanboe, *Complementary filter*, https://seanboe.com/blog/complementary-filters, Accessed: 2025-09-22, 5 July 2021.

[2] STMicroelectronics, *Benefits of using fifo buffer*, https://www.st.com/resource/en/design_tip/dt0011-benefits-in-using-fifo-buffer-embedded-in-st-mems-sensors-stmicroelectronics.pdf, Accessed: 2025-09-22, 2013.

[3] Microsoft, *Sensor data batching for power savings*, https://learn.microsoft.com/en-us/windows-hardware/drivers/sensors/sensor-batching-for-power-saving-, Accessed: 2025-10-2, 11. Oct. 2024.

[4] Android Open Source Project, *Batching*, https://source.android.com/docs/core/interaction/sensors/batching, Describes buffering sensor events to save power and the maximum reporting latency (max delay). Accessed: 2025-10-02, 2024.

[5] T. InvenSense, *Icm-42688-p: 6-axis mems motiontracking device*, https://product.tdk.com/system/files/dam/doc/product/sensor/mortion-inertial/imu/data_sheet/ds-000347-icm-42688-p-v1.6.pdf, Includes FIFO timestamp details enabling Δt reconstruction. Accessed: 2025-10-02, 2021.

[6] STMicroelectronics, *Lsm6ds3tr-c: Always-on 3d accelerometer and 3d gyroscope*, https://www.st.com/resource/en/application_note/an5130-lsm6ds3trc-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf, Application Note, Smart FIFO with optional timestamp; batching. Accessed: 2025-10-02, 2018.

[7] BMI270, *Bmi270*, https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi270-ds000.pdf, Accessed: 2025-10-2, march 2023.

[8] LIS2DH, *Lis2dh*, https://semic.cz/!old/files/pdf_www/LIS2DH_ST.pdf, Accessed: 2025-10-2, nov. 2011.

[9] LSM6DSOX, *Lsm6dsox*, https://www.st.com/resource/en/datasheet/lsm6dsox.pdf&ved=2ahUKEwic8dmcu4WQAxX63QIHHTevGBcQFnoECAoQAQ&usg=AOvVaw3yto92UevKnnlUg845zeDh, Accessed: 2025-10-2, jun. 2024.

[10] Bosch, *Bmi270 api*, https://github.com/boschsensortec/BMI270_SensorAPI?, Accessed: 2025-10-2, jun. 2023.

[11] FLUKE, *Fluke 289 true rms multimeter*, https://www.fluke-direct.com/pdfs/cache/www.fluke-direct.com/289/datasheet/289-datasheet.pdf, Accessed: 2025-10-2, jun. 2023.

[12] picoscope, *Picoscope 2408b*, https://www.batronix.com/files/Pico/PicoScope2000/Downloads/picoscope-2000-series-data-sheet-de.pdf, Accessed: 2025-10-2, 23 may 2021.

Appendix A

A.0.1 The external device with the headpiece.

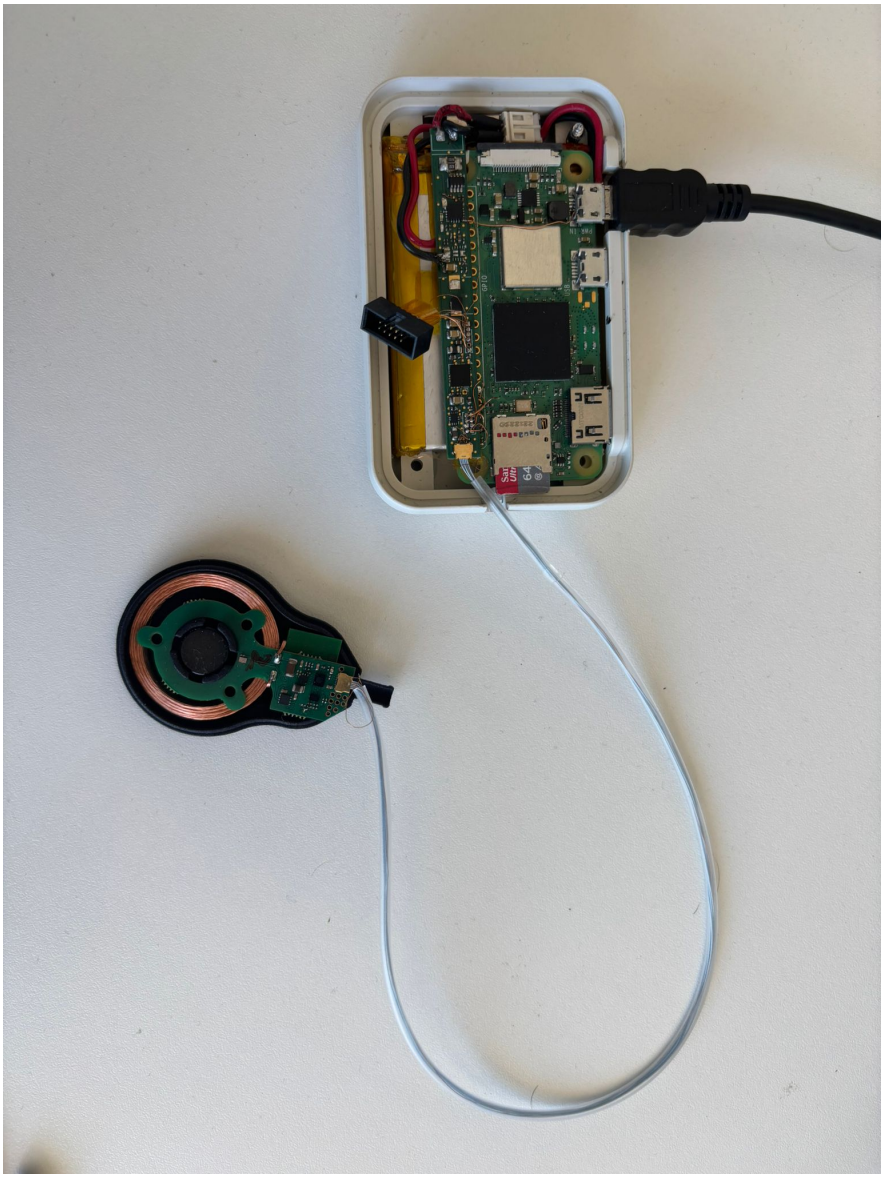


Figure A.1: external device

A.0.2 Live data for emulating arm movement

Figures A.2 and A.3 show two live runs with comparable hand motion. Blue: estimated pitch angle. Red: gyroscope rate. With the complementary filter (Fig. A.2), the angle is smooth and stable while still tracking rapid changes. Without the filter (Fig. A.3, accelerometer-only baseline), the angle is visibly noisier and less consistent. Both tests were performed on the LSM6DSOX sensor.

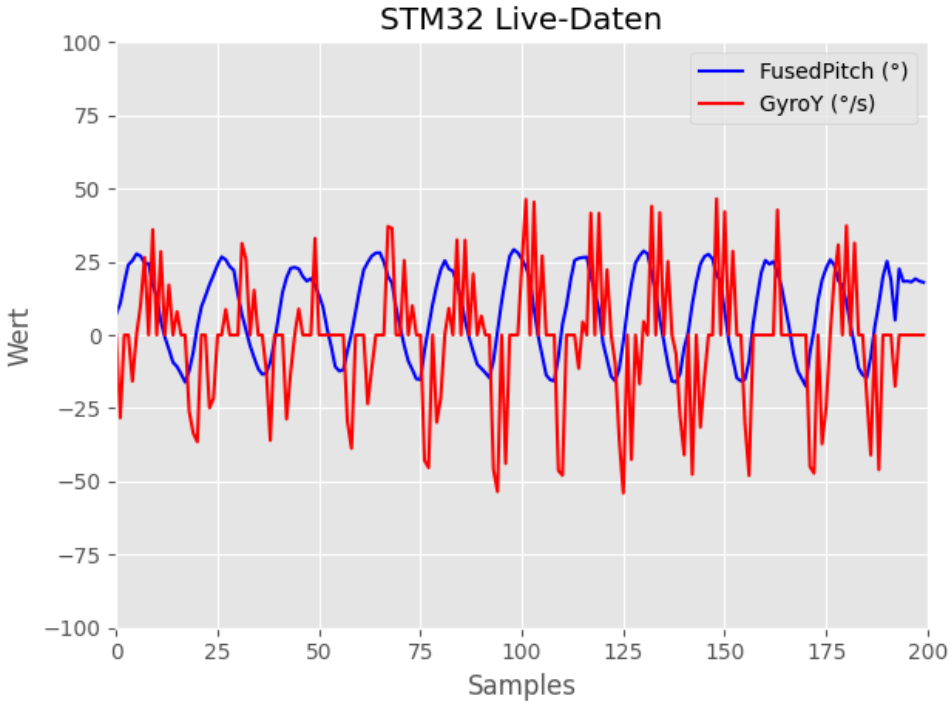


Figure A.2: Live stream with complementary filter.

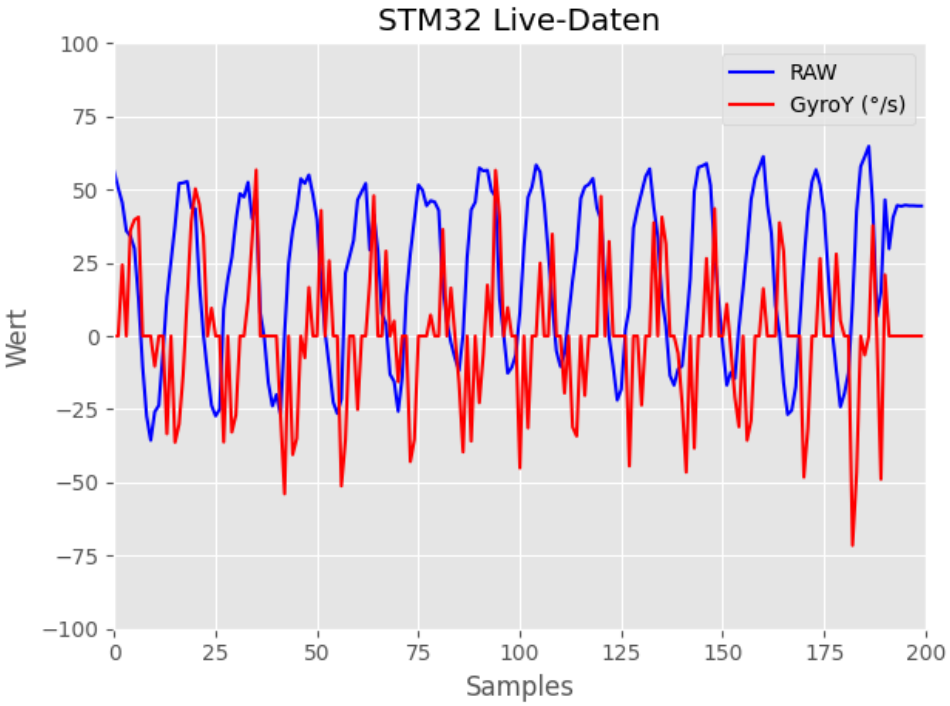


Figure A.3: Live stream without complementary filter.